

STUDENT MANAGEMENT SYSTEM

CRUD-Based Full Stack Web Application

Project Documentation

Technology: HTML, CSS, JavaScript, Django REST Framework & SQLite

Academic Full-Stack Web Development Project

Prepared for project demonstration and submission

1. Project Overview

The Student Management System is a full-stack CRUD web application developed to manage student records efficiently. It provides a simple dashboard through which users can add new students, view stored records, edit existing information, delete records, and search the student list. The project demonstrates frontend development, backend REST APIs, database integration, validation, testing, and documentation in one working application.

2. Problem Statement

Manual maintenance of student information can lead to duplicated data, slow searching, inconsistent records, and difficulty in updating information. A centralized web application is required to store and manage student details in a structured database while providing a simple interface for common data-management operations.

3. Objectives

The main objectives are to develop a responsive student-management interface; implement Create, Read, Update, and Delete operations; build RESTful API endpoints using Django REST Framework; store records persistently using SQLite; validate student data; provide search functionality; handle application errors clearly; and document the setup, testing, and execution process.

4. Technology Stack

Layer	Technology	Purpose
Frontend	HTML5, CSS3, JavaScript	Responsive user interface and client-side interaction
Backend	Python Django + Django REST Framework	Server-side logic and REST API
Database	SQLite	Persistent student-data storage
API Format	JSON / HTTP	Frontend-backend data exchange
Version Control	GitHub	Source-code hosting and project submission

5. System Architecture

The application follows a layered architecture. The user interacts with the HTML/CSS/JavaScript frontend. JavaScript sends HTTP requests to the Django REST API. Django processes the request, applies validation and business logic, and communicates with SQLite through the Django ORM. The response is returned as JSON and the frontend updates the interface dynamically.

Architecture Flow

User -> Web Frontend -> REST API -> Django Backend -> Django ORM -> SQLite Database

6. Main Student Entity

The main entity is Student. Typical fields used by the application include Student ID, Name, Email, Department, and Year. Student ID and email are validated to avoid inappropriate or duplicate values, and the year value is restricted to an acceptable academic range.

7. CRUD Implementation

Create: The user opens the Add Student form, enters valid student information, and submits it. The frontend sends a POST request and the new record is saved in SQLite.

Read: The dashboard requests student records using GET and displays them in a responsive table.

Update: The Edit control loads an existing record. After modification, the frontend sends an update request and the table shows the new values.

Delete: The Delete control removes the selected record after confirmation and refreshes the displayed data.

8. REST API Endpoints

Operation	Method	Endpoint	Result
Create	POST	/api/students/	Creates a student
Read All	GET	/api/students/	Returns all students
Read One	GET	/api/students/{id}/	Returns one student
Update	PUT/PATCH	/api/students/{id}/	Updates a student
Delete	DELETE	/api/students/{id}/	Deletes a student

9. Validation and Error Handling

The application validates required fields before accepting records. Email values are checked for valid formatting, unique fields are protected from duplicates, and the academic year is restricted to the expected range. Backend validation remains authoritative even when client-side checks are available. The interface displays success or error feedback after operations.

10. Search Functionality

The dashboard includes a search field that allows users to quickly locate records by values such as Student ID, student name, or department. This improves usability when the number of stored students increases.

11. Testing

The project was tested for Create, Read, Update, Delete, validation, and search operations. Database migrations were applied successfully and the application was demonstrated with sample student records. API behavior was verified for normal CRUD operations, and the working application was run locally through the Django development server.

Testing Summary

Test	Expected Result	Status
Create valid student	New record saved and displayed	Passed

Test	Expected Result	Status
Read student list	Stored records displayed	Passed
Update record	Edited values displayed	Passed
Delete record	Selected record removed	Passed
Validation	Invalid/duplicate data handled	Passed
Search	Matching students filtered/displayed	Passed

12. Project Structure

The repository contains the Django configuration folder **student_config**, the main **students** application, database migration files, **manage.py**, **requirements.txt**, **README.md**, **TESTING.md**, **.gitignore**, and the SQLite database used for the demonstration.

13. Installation and Execution

1. Install Python on the computer.
2. Open a terminal in the project directory.
3. Install dependencies using **pip install -r requirements.txt**.
4. Run database migrations using **python manage.py migrate**.
5. Start the application using **python manage.py runserver**.
6. Open **http://127.0.0.1:8000/** in a browser.

14. Security and Quality Practices

Secrets and credentials should not be hard-coded. Input should be validated on the server, database operations should use Django ORM, temporary environments and cache files should be excluded from version control, and frontend, backend, and database responsibilities should remain separated.

15. Challenges and Solutions

A key practical challenge was completing frontend-backend integration and ensuring the interface refreshed correctly after CRUD operations. This was handled by using JavaScript API requests and dynamically updating the displayed records. Another challenge was preparing the project for submission; unnecessary virtual-environment and dependency folders were excluded while the essential source code and documentation were retained.

16. Future Enhancements

Future versions can add user login and role-based authorization, pagination, advanced filtering, student profile images, export to CSV/PDF, cloud database deployment, audit logs, and a React-based frontend.

17. Conclusion

The Student Management System successfully demonstrates a complete CRUD-based full-stack workflow. It integrates a responsive frontend, Django REST backend, RESTful communication, validation, SQLite persistence, search, testing, documentation, and GitHub-based source-code

management. The project satisfies the core completion criteria of the provided SOP and can be demonstrated through working Create, Read, Update, and Delete operations.

18. Viva - Quick Explanation

What is CRUD? Create, Read, Update and Delete are the four basic operations used to manage database records.

Why Django REST Framework? It provides tools for building REST APIs in Python and makes it easier to serialize, validate, retrieve, and modify database data.

Why SQLite? It is lightweight, requires minimal configuration, and is suitable for an academic demonstration project.

How does the frontend communicate with the backend? JavaScript sends HTTP requests to REST API endpoints and receives JSON responses.

Where is data stored? Student records are stored persistently in the SQLite database through Django ORM.